

Validación de sprites del videojuego



Figura 1. Organización de sprites dentro del entorno de desarrollo GameMaker.

En esta sección se presentan los sprites utilizados en el desarrollo del videojuego educativo. Los sprites corresponden a todos los elementos visuales que hacen parte del entorno del juego, incluyendo personajes, objetos de interacción, elementos del escenario y recursos gráficos utilizados en los minijuegos.

Durante las pruebas, se verificó que todos los sprites se encuentran correctamente organizados dentro del entorno de desarrollo (GameMaker), agrupados según su funcionalidad, como NPC, jugador, objetos de interacción y elementos visuales del mapa. Esta organización permite una mejor gestión del proyecto y facilita la implementación de cada componente dentro del sistema.

Se evidencia la presencia de sprites correspondientes a:

- Personajes principales y NPC (como Joseph, Liza, Milan, Mimi, entre otros)
- Elementos del jugador (sprites del personaje controlable)
- Objetos de interacción (mesas, sillas, botones, televisión, etc.)
- Elementos del entorno (árboles, plantas, cuadros, mapas)
- Barreras y límites del escenario (colisiones)
- Recursos visuales utilizados en minijuegos (bloques, fichas, direcciones)

La correcta carga de estos recursos garantiza que el videojuego pueda representar visualmente cada uno de los elementos definidos en la fase de diseño, permitiendo la interacción del jugador con el entorno.

Además, se comprobó que los sprites pueden ser utilizados sin errores dentro del sistema, lo que indica que:

- Están correctamente importados
- Tienen nombres coherentes
- Se encuentran disponibles para su uso en objetos y escenas

Validación de escenarios (Rooms) del videojuego

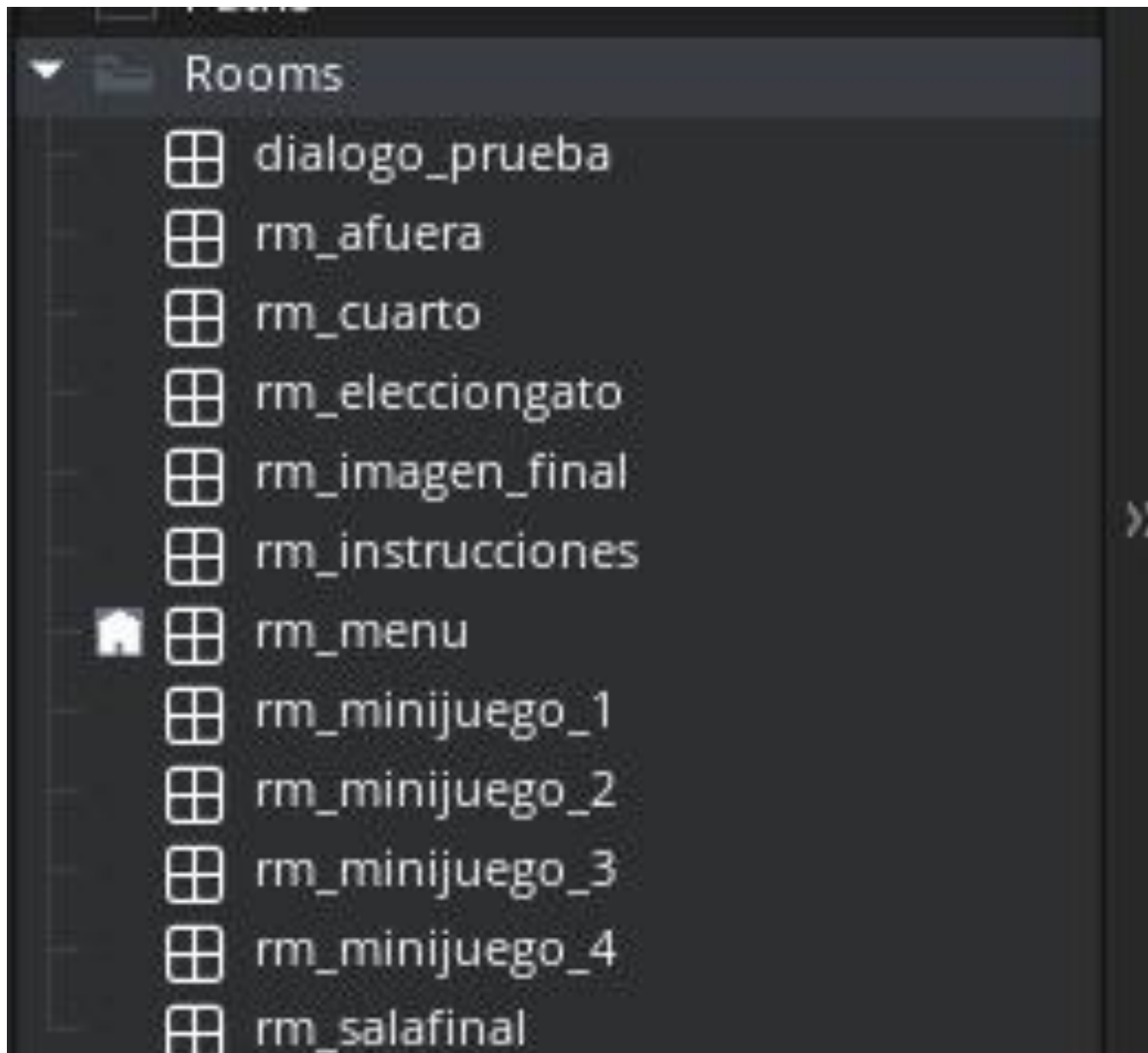


Figura 2. Organización de escenarios (rooms) dentro del entorno de desarrollo GameMaker.

En esta sección se presentan los escenarios del videojuego, conocidos como *rooms* dentro del entorno de desarrollo GameMaker. Cada escenario representa un espacio específico del juego en el que se desarrolla una parte de la experiencia del usuario, ya sea navegación, interacción, aprendizaje o ejecución de minijuegos.

Durante las pruebas, se verificó que los escenarios están correctamente definidos y organizados, evidenciando una estructura lógica que guía al jugador a lo largo del videojuego. Se identifican diferentes tipos de escenarios, cada uno con una función clara dentro del sistema:

- **Escenarios de navegación principal**, como el menú (*rm_menu*), que permite al usuario iniciar o acceder a las diferentes opciones del juego.
- **Escenarios informativos**, como instrucciones (*rm_instrucciones*) y diálogos (*dialogo_prueba*), donde se presentan explicaciones necesarias para el aprendizaje.
- **Escenarios de exploración**, como exteriores e interiores (*rm_afuera*, *rm_cuarto*), donde el jugador se desplaza e interactúa con el entorno y los NPC.
- **Escenarios de minijuegos**, organizados en diferentes niveles (*rm_minijuego_1*, *rm_minijuego_2*, *rm_minijuego_3*, *rm_minijuego_4*), los cuales corresponden a los distintos conceptos de programación trabajados en el videojuego.
- **Escenarios de transición y finalización**, como selección (*rm_elecciongato*), pantalla final (*rm_imagen_final*) y cierre del juego (*rm_salafinal*), que permiten dar continuidad y finalización a la experiencia del usuario.

La correcta organización de estos escenarios permite validar que el videojuego cuenta con una navegación estructurada, donde cada etapa está claramente definida y conectada con las demás. Esto garantiza que el usuario pueda avanzar de manera progresiva, pasando de la explicación de conceptos a su aplicación práctica en los minijuegos.

Además, se comprobó que los escenarios pueden cargarse correctamente durante la ejecución del juego, lo que indica que:

- Están correctamente configurados dentro del motor
- Permiten la transición entre estados del juego
- Soportan la integración de personajes, objetos y lógica de interacción

Validación de objetos del videojuego

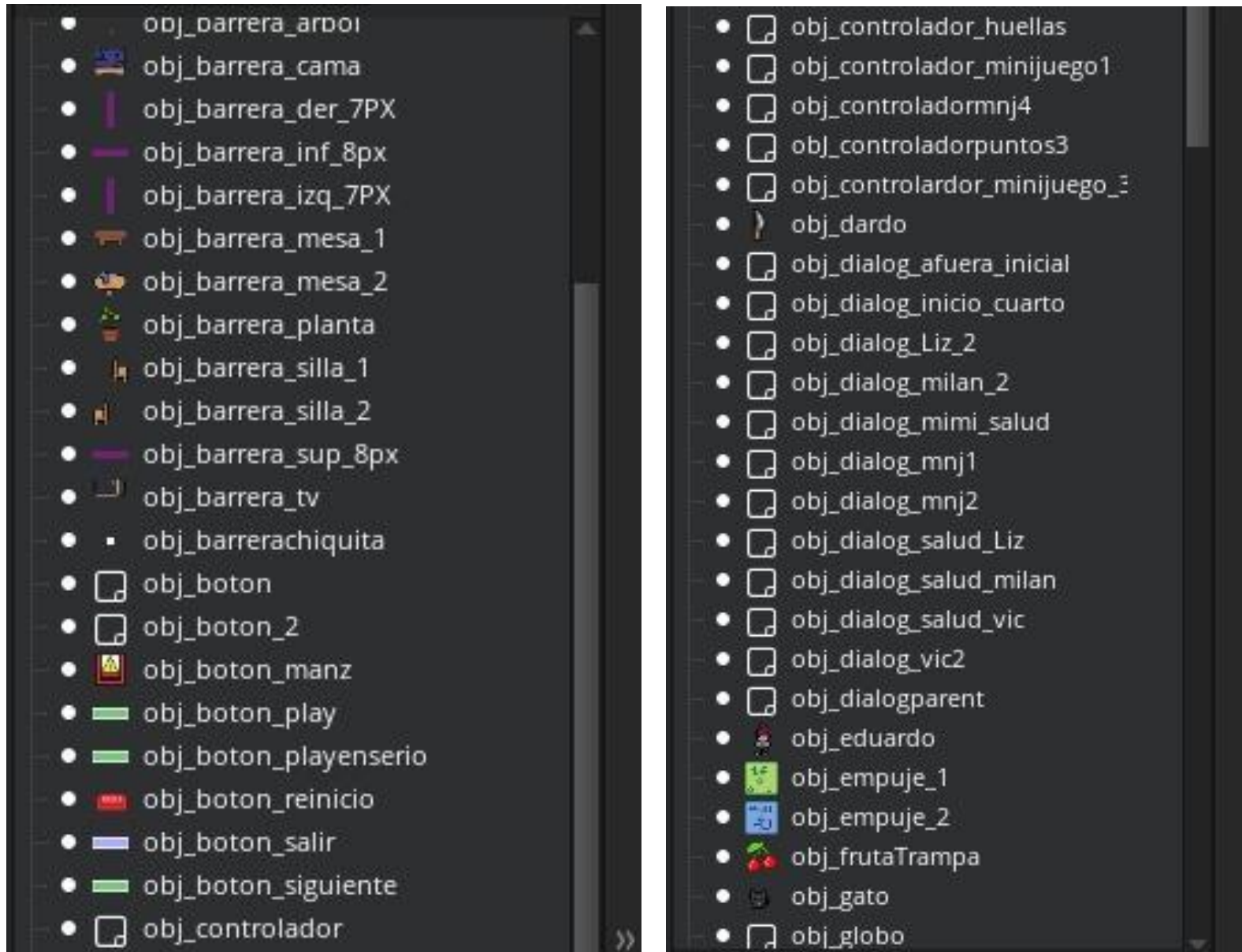


Figura 3. Organización de objetos dentro del entorno de desarrollo GameMaker.

En esta sección se presentan los objetos del videojuego, los cuales representan los elementos funcionales dentro del sistema. A diferencia de los sprites, que corresponden a la parte visual, los objetos son los encargados de definir el comportamiento del juego, incluyendo interacciones, controles, lógica de los minijuegos y respuestas del sistema.

Durante las pruebas, se verificó que los objetos están correctamente organizados dentro del entorno de desarrollo, evidenciando una estructura clara según su funcionalidad. Se identifican diferentes tipos de objetos que cumplen roles específicos dentro del videojuego:

- **Objetos de control y lógica**, como controladores del sistema (*obj_controlador*, *obj_controladormnj*, entre otros), que gestionan el flujo del juego, la ejecución de eventos y el comportamiento general del sistema.
- **Objetos de interfaz**, como botones (*obj_boton_play*, *obj_boton_salir*, *obj_boton_reinicio*, *obj_boton_siguiente*), que permiten la interacción directa del usuario con el juego.
- **Objetos de colisión o barreras**, utilizados para limitar el movimiento del jugador dentro del escenario (*obj_barrera_*, *obj_barre...**), garantizando que el desplazamiento se mantenga dentro de las zonas definidas.
- **Objetos de interacción y entorno**, como mesas, sillas, televisión y otros elementos del escenario, que enriquecen la experiencia del jugador y permiten la simulación de un entorno más realista.
- **Objetos de diálogo**, que controlan la presentación de mensajes e instrucciones (*obj_dialog_*), fundamentales para el componente educativo del videojuego.
- **Objetos de minijuegos**, como globos, gatos, frutas y elementos dinámicos (*obj_globo*, *obj_gato*, *obj_frutaTrampa*), que permiten ejecutar la lógica específica de cada actividad dentro del juego.

La correcta implementación de estos objetos permite validar que el sistema puede:

- Gestionar las acciones del jugador
- Ejecutar eventos e interacciones dentro del entorno
- Controlar la lógica de los minijuegos
- Mostrar información y retroalimentación al usuario

Adicionalmente, se comprobó que los objetos responden correctamente a los eventos programados (colisiones, clics, movimientos, activaciones), lo que garantiza su correcto funcionamiento dentro del videojuego.

Validación completa de la programación del personaje principal

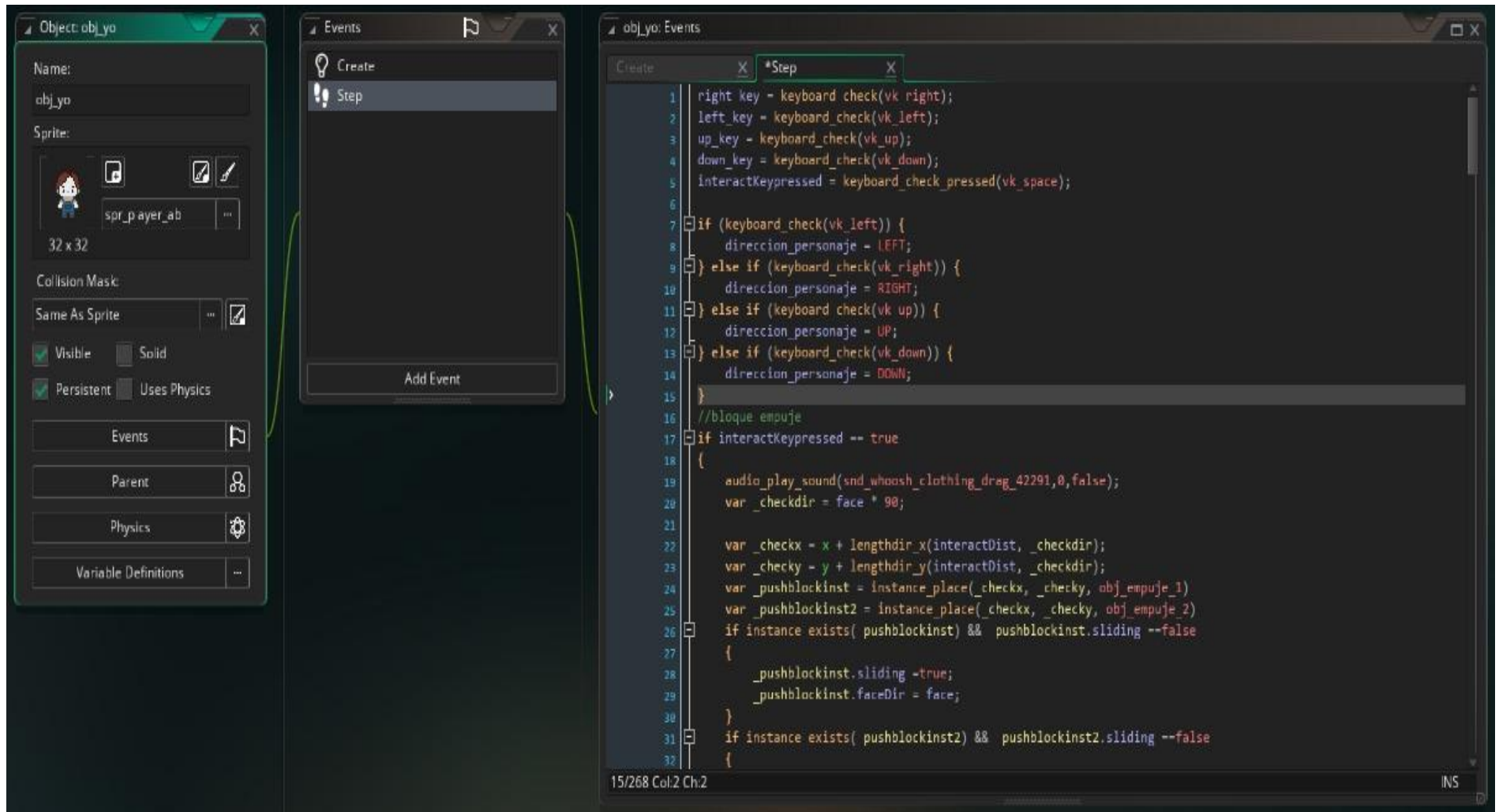


Figura 4. Configuración del objeto del personaje principal en GameMaker.

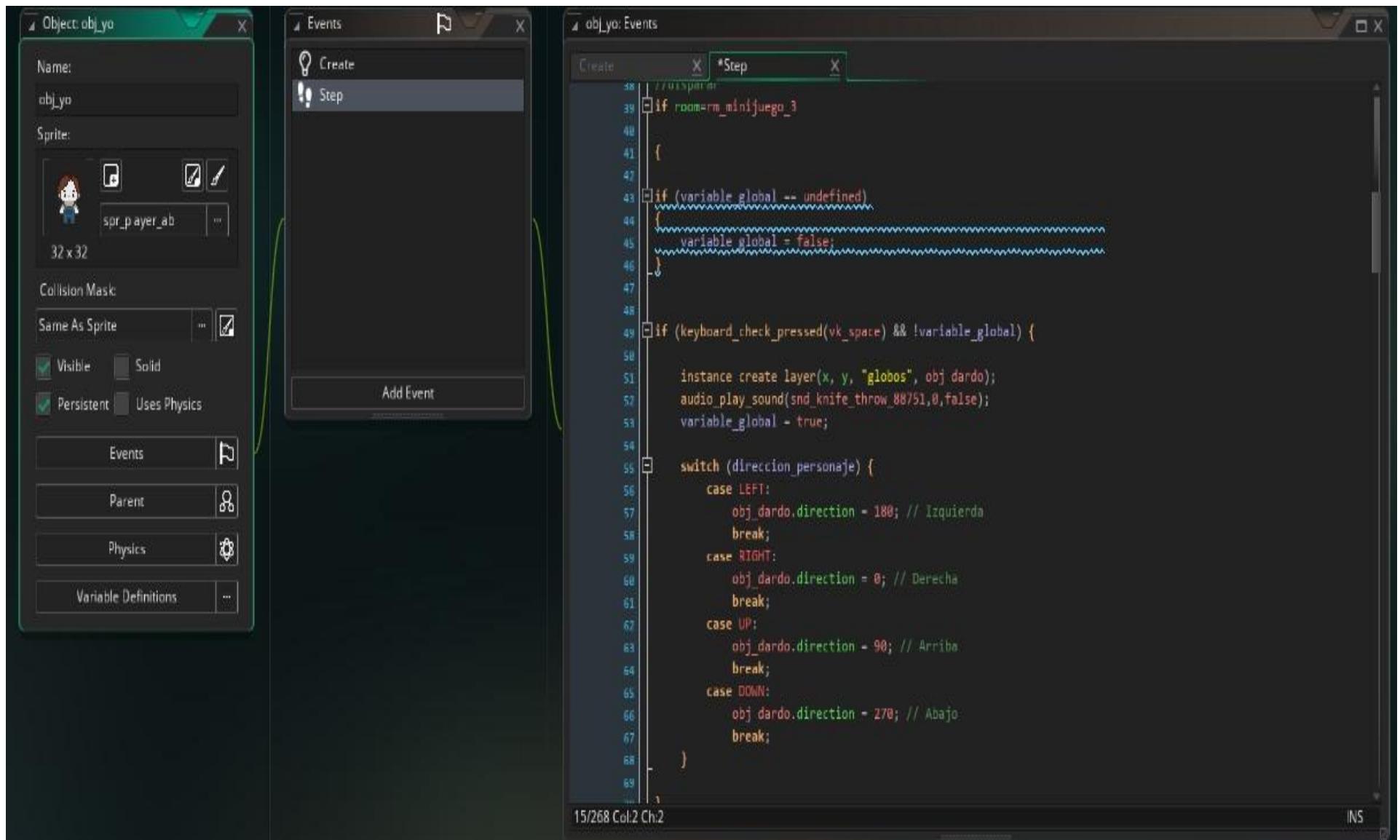


Figura 5. Implementación del movimiento e interacción del personaje en el evento Step.

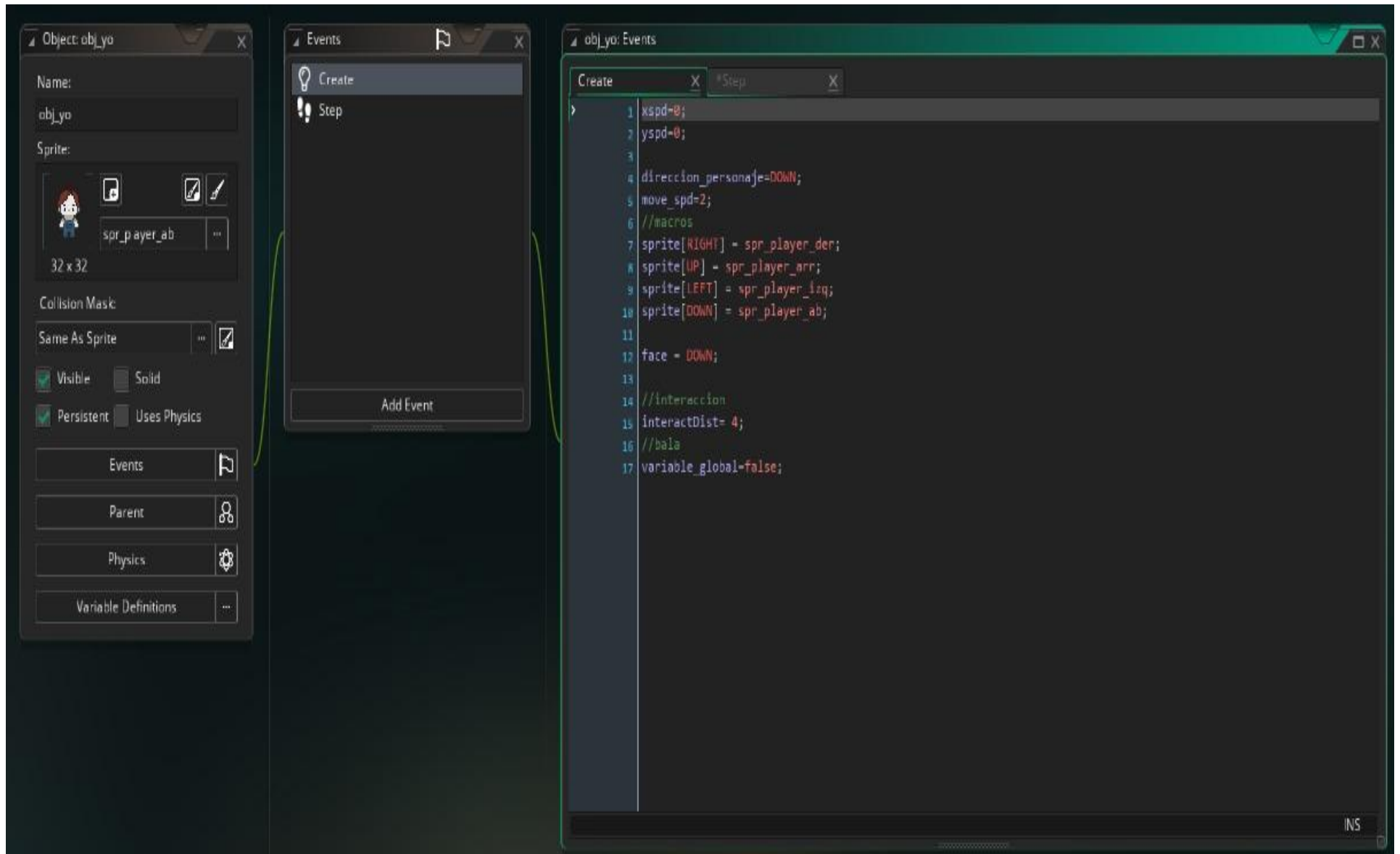


Figura 6. Inicialización de variables y configuración base del personaje en el evento Create.

En esta sección se presenta la implementación del comportamiento del personaje principal dentro del videojuego, desarrollada en el entorno de GameMaker mediante eventos como *Create* y *Step*. Esta programación define no solo el movimiento del jugador, sino también su interacción con el entorno y la ejecución de acciones dentro de los minijuegos.

Inicialmente, en el evento de creación (*Create*), se configuran las variables base del personaje, como la velocidad de movimiento, la dirección inicial, los sprites correspondientes a cada orientación (izquierda, derecha, arriba y abajo) y variables auxiliares utilizadas para la interacción. Esto permite establecer un estado inicial controlado para el personaje dentro del sistema.

Posteriormente, en el evento de ejecución continua (*Step*), se implementa la lógica de control del jugador mediante la detección de teclas del teclado. Se verifica que:

- El personaje responde correctamente a las teclas direccionales (W, A, S, D o flechas)
- Se actualiza la dirección del personaje en función de la entrada del usuario
- Se asigna el sprite correspondiente según la orientación del movimiento

Adicionalmente, se implementa una mecánica de interacción mediante una tecla específica (como la barra espaciadora), la cual permite ejecutar acciones dentro del entorno. Esta interacción incluye:

- Activación de eventos cercanos
- Reproducción de sonidos
- Comunicación con otros objetos del sistema

El código también incorpora lógica para la detección de objetos cercanos utilizando cálculos de posición, lo que permite identificar elementos con los que el jugador puede interactuar. Esto es fundamental para la ejecución de mecánicas como empujar objetos o activar eventos dentro del mapa.

Por otra parte, se evidencia la implementación de funcionalidades adicionales como el lanzamiento de objetos (por ejemplo, proyectiles o acciones dentro de minijuegos), donde el sistema:

- Crea nuevas instancias dinámicas dentro del entorno
- Define su dirección en función del movimiento del personaje
- Ejecuta acciones específicas según el contexto del juego

Durante las pruebas se comprobó que:

- El personaje se mueve correctamente en todas las direcciones
- La animación y orientación coinciden con la acción realizada
- Las interacciones se activan correctamente mediante la tecla definida
- La detección de objetos cercanos funciona sin errores
- La creación de instancias (como proyectiles o acciones dinámicas) se ejecuta correctamente
- No se presentan errores críticos en la ejecución del código

Validación de la programación de los enemigos (ratones) en el minijuego de arrays

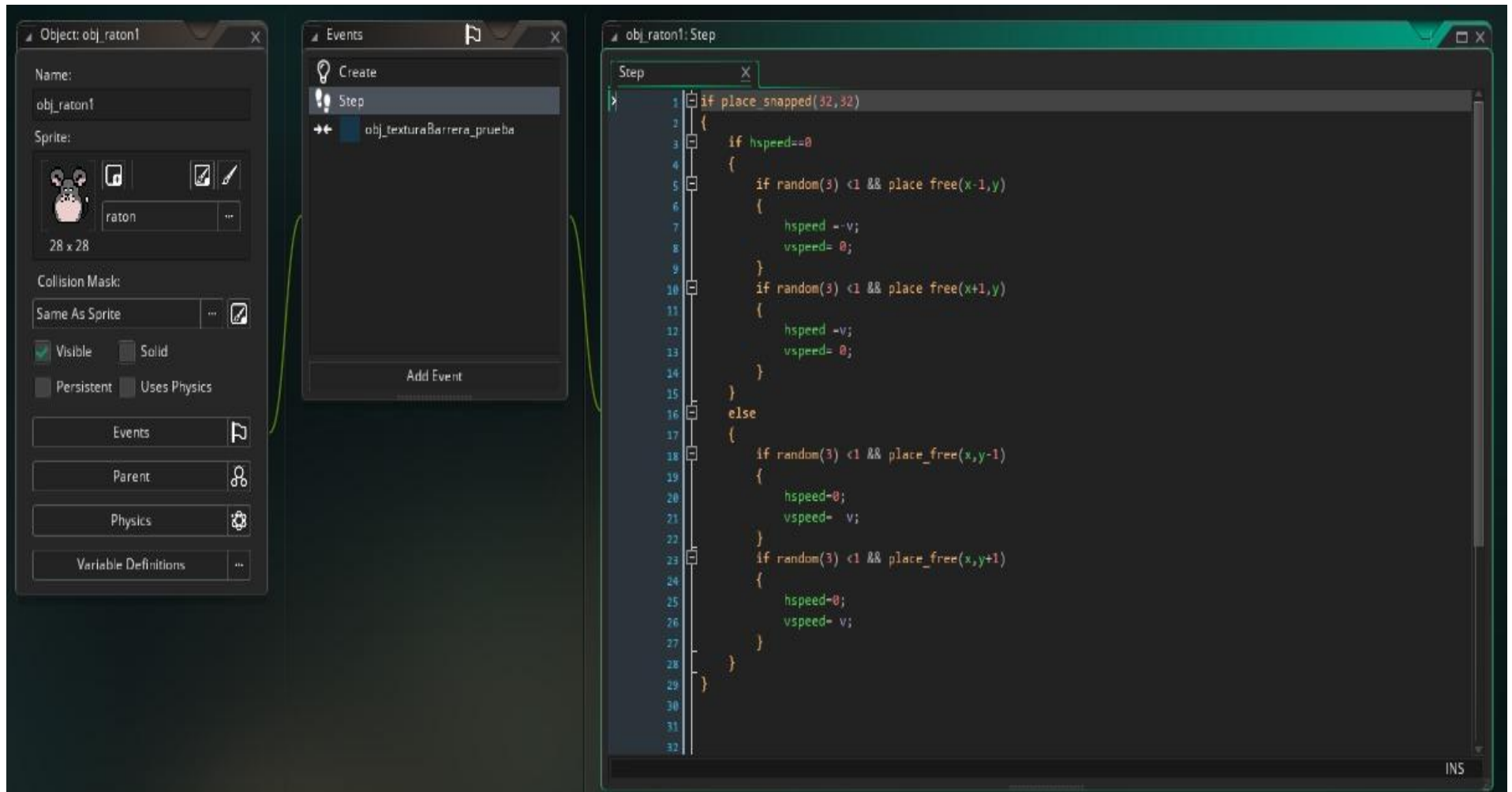


Figura 7. Programación del comportamiento de los enemigos (ratones) en el minijuego de arrays.

En esta sección se presenta la programación de los enemigos del minijuego de arrays, representados por objetos tipo “ratón”. Estos elementos cumplen un papel fundamental dentro de la dinámica del juego, ya que introducen un nivel de dificultad al obligar al jugador a tomar decisiones rápidas y correctas durante la recolección de elementos.

La lógica implementada para estos enemigos se desarrolla en el evento de ejecución continua (*Step*), donde se define su comportamiento dentro del escenario. Se evidencia que los ratones se desplazan de manera autónoma, utilizando condiciones aleatorias que determinan su dirección de movimiento.

El código muestra el uso de funciones como `random()` y `place_free()`, lo cual permite que los enemigos:

- Selecciones direcciones de movimiento de forma aleatoria
- Eviten atravesar obstáculos o zonas no permitidas
- Se desplacen únicamente en espacios válidos del mapa

Esto genera un comportamiento dinámico, donde los enemigos no siguen un patrón fijo, sino que se mueven de manera impredecible dentro del escenario, aumentando la complejidad del minijuego.

Además, se controla la velocidad del movimiento mediante variables como `hspeed` y `vspeed`, lo que permite ajustar la dificultad del juego dependiendo del comportamiento deseado.

Durante las pruebas se verificó que:

- Los enemigos se mueven de forma autónoma sin intervención del jugador
- No atraviesan obstáculos ni zonas bloqueadas
- Cambian de dirección de manera aleatoria
- Mantienen un comportamiento continuo dentro del escenario
- No generan errores durante su ejecución

Este comportamiento es clave dentro del minijuego de arrays, ya que obliga al jugador a identificar correctamente los elementos del arreglo mientras evita o se adapta al movimiento de los enemigos.

Validación de la programación de los globos en el minijuego de variables

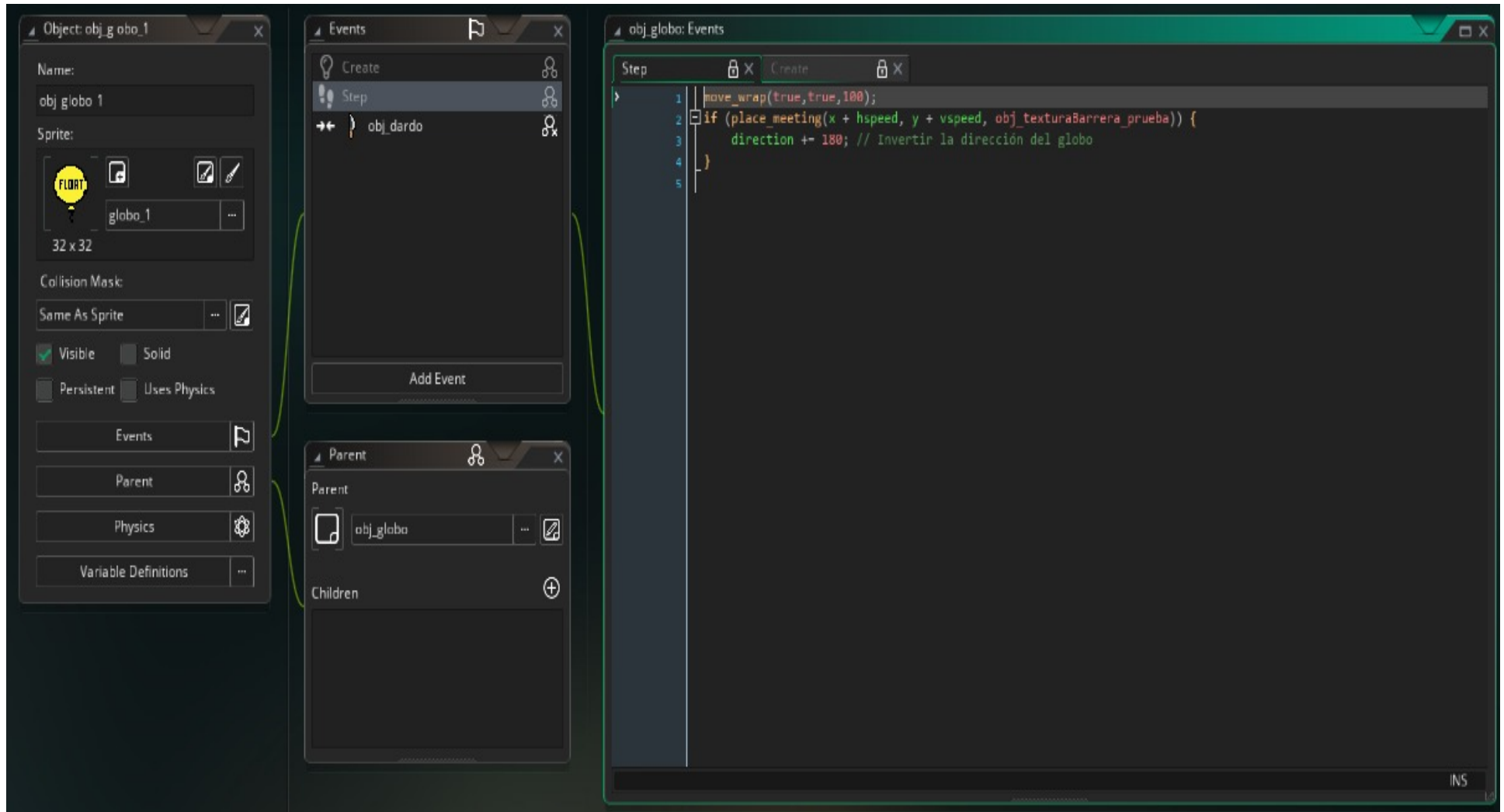


Figura 8. Implementación del movimiento y detección de colisiones de los globos en el minijuego de variables.

En esta sección se presenta la programación de los globos utilizados en el minijuego de tipos de variables. Estos elementos representan visualmente los distintos tipos de datos dentro del juego y constituyen el medio principal de interacción para el usuario durante esta actividad.

La lógica implementada para los globos se desarrolla mediante eventos dentro de GameMaker, donde se define su comportamiento dinámico en el entorno. Se evidencia que los globos se desplazan de manera continua dentro del escenario utilizando funciones como `move_wrap()`, lo que permite que se mantengan en movimiento constante sin salir de los límites del mapa.

Adicionalmente, se implementa una condición de colisión mediante la función `place_meeting()`, la cual permite detectar cuando el globo entra en contacto con un objeto del entorno (como barreras). En estos casos, el sistema invierte la dirección del movimiento, generando un comportamiento dinámico y controlado dentro del espacio de juego.

Este tipo de programación permite que los globos:

- Se muevan de forma autónoma dentro del escenario
- Reaccionen ante los límites del entorno
- Mantengan un flujo constante de movimiento
- Generen interacción activa con el jugador

Durante las pruebas realizadas, se verificó que:

- Los globos se desplazan correctamente dentro del escenario
- No atraviesan barreras u obstáculos
- Cambian de dirección al colisionar con límites del entorno
- Mantienen un comportamiento continuo sin errores
- Permiten la interacción directa con el jugador durante el minijuego

Este comportamiento es fundamental dentro del minijuego de variables, ya que obliga al jugador a identificar correctamente los tipos de datos mientras interactúa con elementos en movimiento, reforzando el aprendizaje de forma visual y dinámica.

Validación de la programación de los bloques del minijuego de completar código

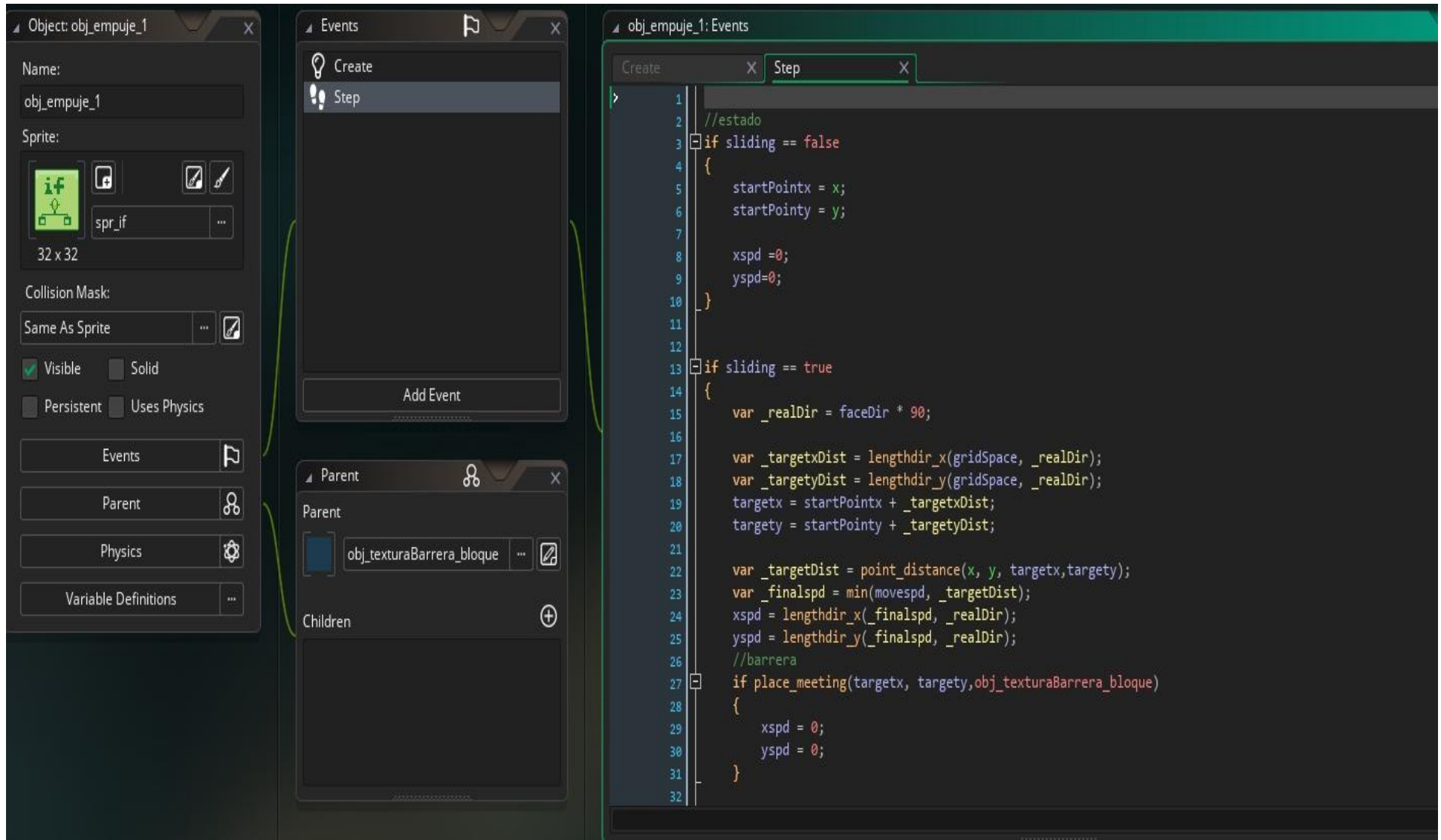


Figura 9. Programación del comportamiento de los bloques en el minijuego de completar código en GameMaker.

En esta sección se presenta la programación de los bloques utilizados en el minijuego de completar código. Estos bloques representan fragmentos de instrucciones que el jugador debe mover o ubicar correctamente para completar una estructura de código dentro del juego.

En la imagen se observa que el objeto del bloque cuenta con eventos de programación en GameMaker, especialmente el evento Step, donde se controla su comportamiento durante la ejecución del minijuego. La lógica permite que el bloque pueda desplazarse hacia una posición objetivo y detenerse cuando llega al lugar indicado.

Durante las pruebas se verificó que los bloques responden correctamente a las condiciones programadas, manteniendo su posición inicial cuando no están activos y desplazándose cuando el sistema lo requiere. También se evidencia el uso de validaciones de posición y colisión, lo que permite controlar que el bloque no atraviese zonas no permitidas y se ubique correctamente dentro del escenario.

Esto es importante porque el minijuego depende de que los fragmentos de código se comporten de forma ordenada y permitan al jugador identificar cuál es el bloque correcto. De esta manera, el sistema ayuda a reforzar la idea de que una instrucción debe estar ubicada correctamente para que el código tenga sentido.

Se comprobó que:

- Los bloques se encuentran correctamente configurados dentro de GameMaker.
- El objeto responde a los eventos definidos.
- La lógica de movimiento funciona según lo esperado.
- El bloque puede detenerse al llegar a una zona determinada.
- No se evidencian errores críticos en su comportamiento.

Validación del objeto controlador del minijuego de algoritmos

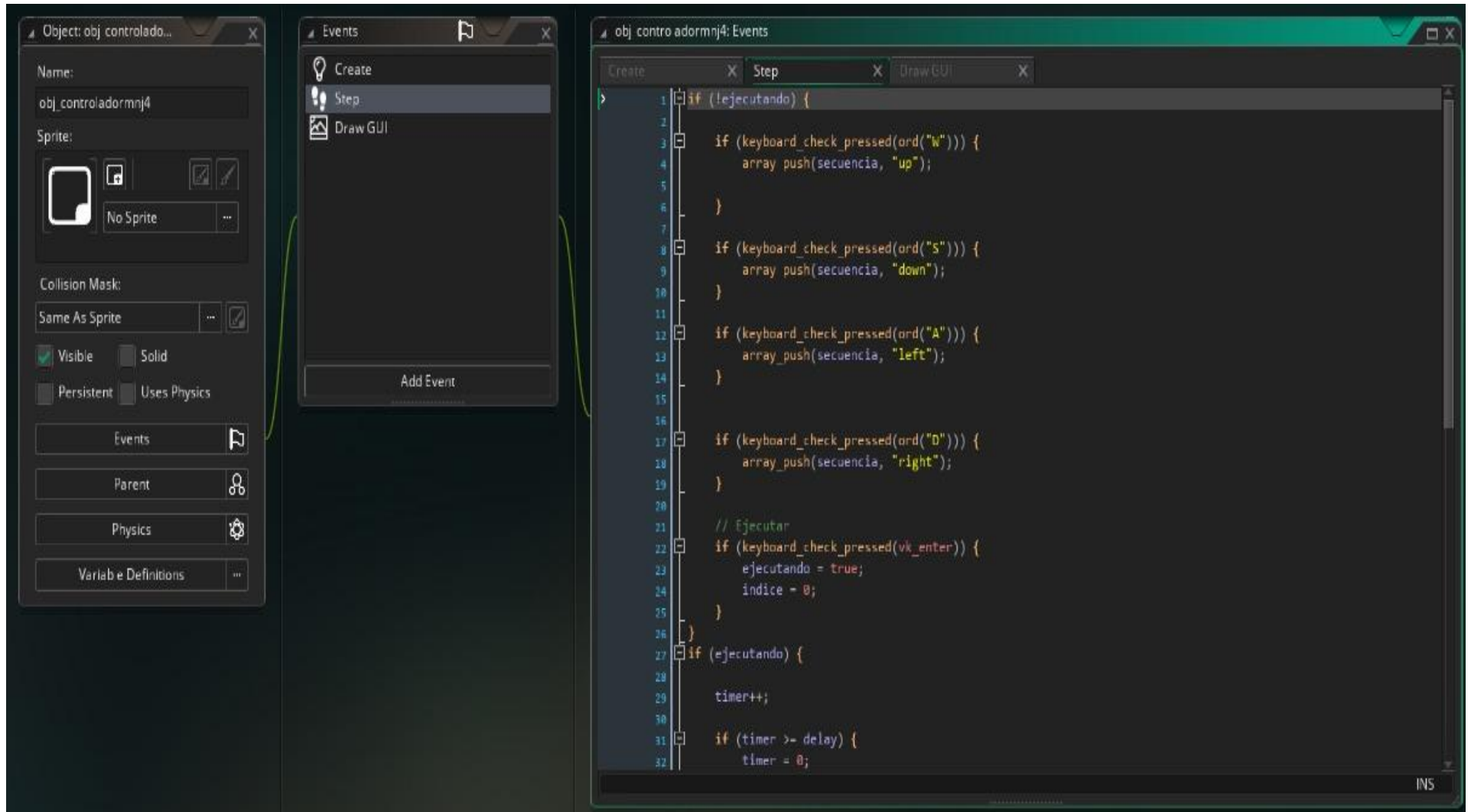


Figura 10. Programación del objeto controlador del minijuego de algoritmos en GameMaker.

En esta sección se presenta la programación del objeto controlador del minijuego 4, correspondiente al minijuego de algoritmos. Este objeto cumple una función central, ya que se encarga de recibir las instrucciones ingresadas por el jugador y controlar la ejecución de la secuencia de movimientos dentro del juego.

En la imagen se observa que el controlador utiliza eventos como Create, Step y Draw GUI, lo que permite inicializar variables, capturar las entradas del teclado y mostrar información en pantalla durante la ejecución del minijuego.

Dentro de la lógica programada, se evidencia que el sistema reconoce instrucciones mediante teclas como W, A, S y D, las cuales representan movimientos del personaje en diferentes direcciones. Cada instrucción ingresada se almacena en una secuencia, permitiendo que el jugador construya paso a paso el recorrido que desea ejecutar.

Posteriormente, al presionar la tecla de ejecución, el sistema activa la secuencia y comienza a procesar los movimientos registrados. Esto permite representar de forma práctica el concepto de algoritmo, ya que el jugador debe planificar una serie de pasos ordenados para resolver el reto.

Durante las pruebas se verificó que:

- El sistema reconoce correctamente las teclas de movimiento.
- Las instrucciones se almacenan en una secuencia.
- El jugador puede ejecutar la secuencia ingresada.
- El controlador gestiona el orden de los movimientos.
- La lógica del minijuego permite trabajar el concepto de algoritmo de forma clara.
- No se evidencian errores críticos en la captura de instrucciones.

Validación del sistema de visualización de puntaje e información en pantalla

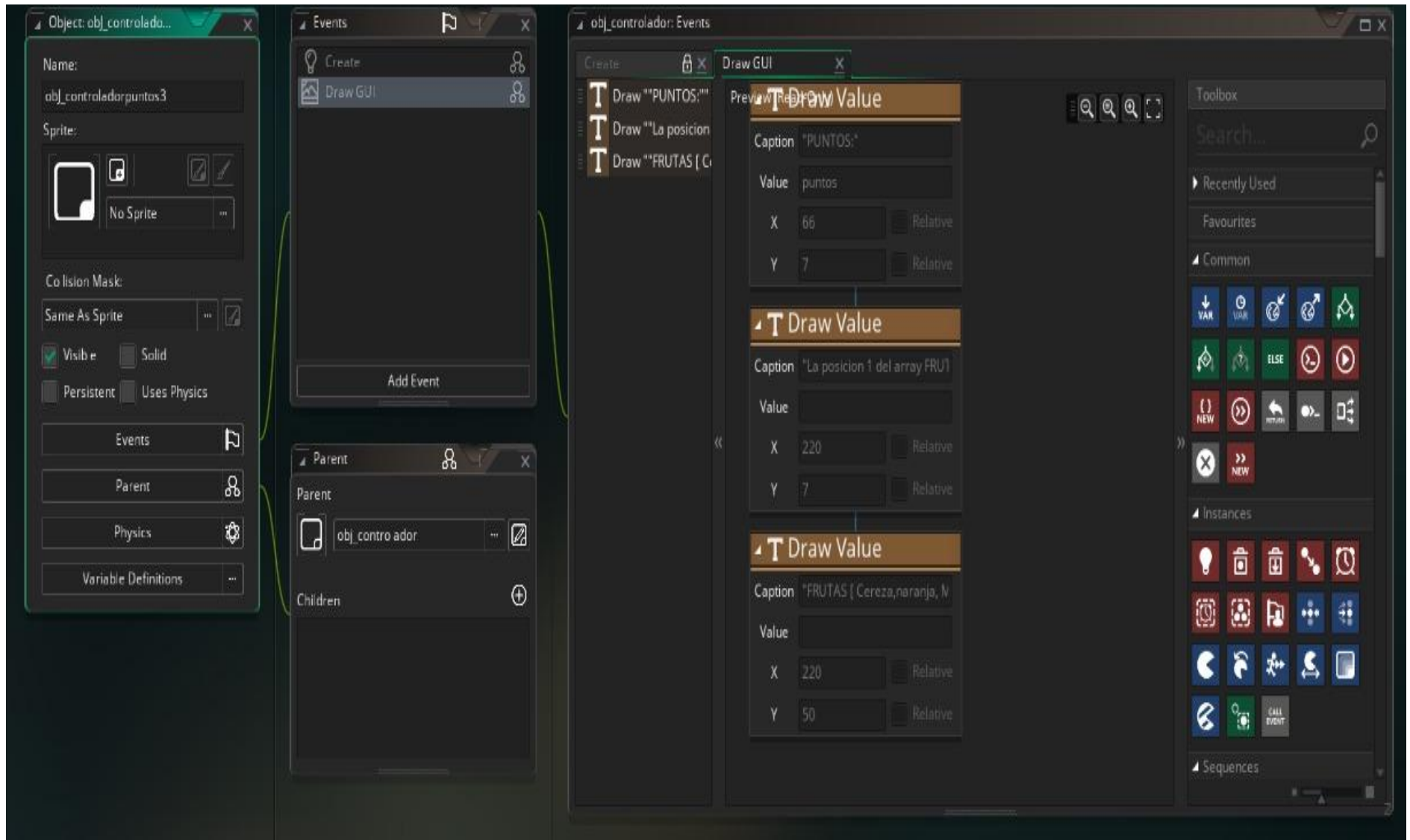


Figura 11. Implementación del sistema de visualización de puntaje e información mediante el evento Draw GUI en GameMaker.

En esta sección se presenta la implementación del sistema encargado de mostrar información en pantalla dentro del videojuego, específicamente los elementos relacionados con el puntaje y datos del estado del juego.

En la imagen se observa la estructura de un objeto controlador que utiliza el evento Draw GUI, el cual permite renderizar texto directamente en la interfaz del usuario, independiente del movimiento del escenario. Esto es fundamental para garantizar que la información mostrada permanezca fija y visible en todo momento.

Se evidencia el uso de funciones como Draw Value, mediante las cuales se presentan distintos datos en pantalla, tales como:

- El puntaje acumulado por el jugador
- Información relacionada con los elementos recolectados
- Indicadores del estado actual del minijuego

Cada uno de estos valores se posiciona estratégicamente en la interfaz, permitiendo que el usuario pueda visualizar de manera clara su progreso y desempeño durante la ejecución del juego.

Durante las pruebas realizadas, se verificó que:

- Los valores se muestran correctamente en pantalla
- La información permanece visible independientemente del movimiento del jugador
- Los datos se actualizan en tiempo real según las acciones realizadas
- No se presentan errores en la visualización de los textos
- La ubicación de los elementos facilita la comprensión del estado del juego

Este sistema es clave dentro del videojuego, ya que permite brindar retroalimentación inmediata al jugador, reforzando el aprendizaje y facilitando la toma de decisiones durante los minijuegos.

Validación de la programación de los diálogos del videojuego

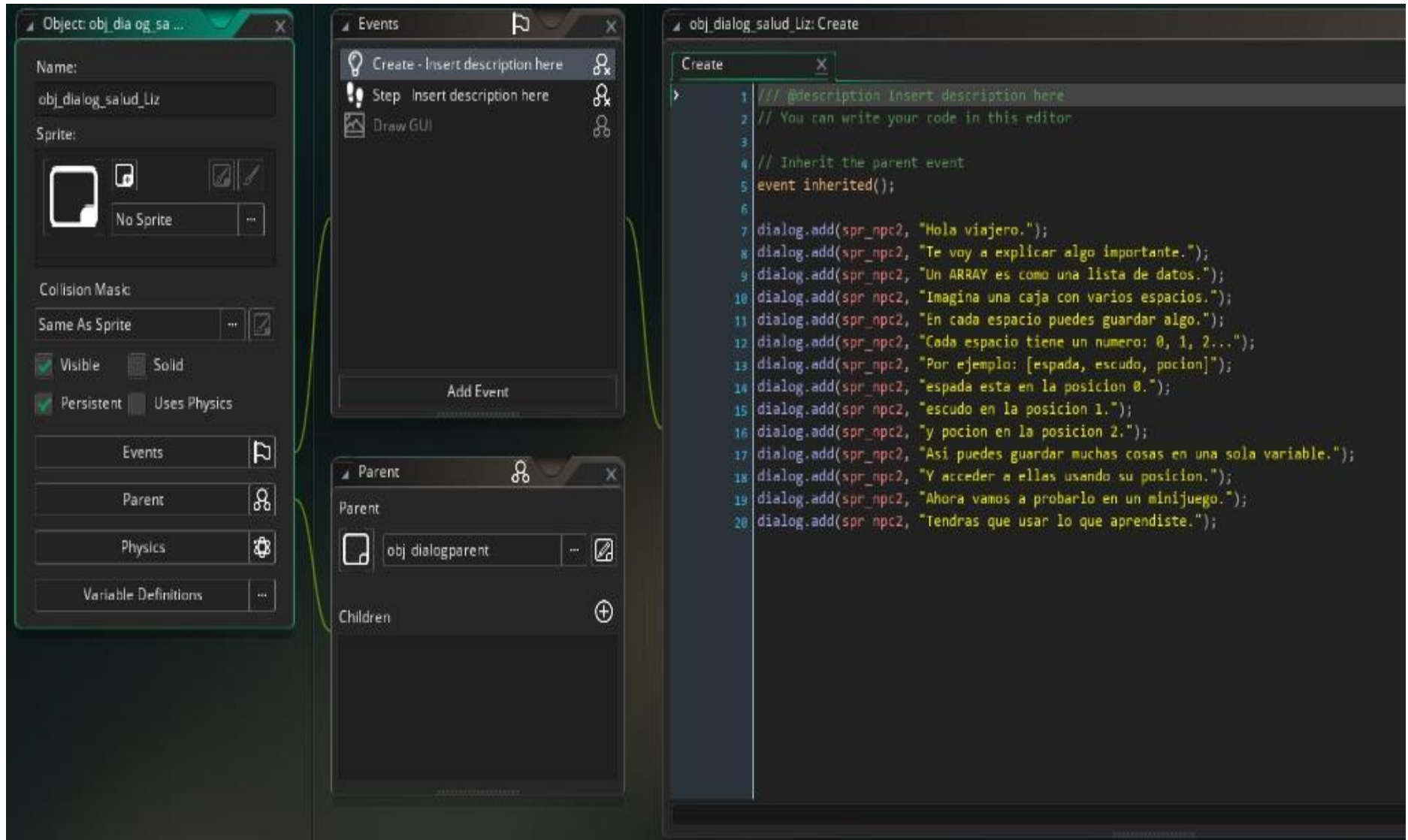


Figura 12. Programación del sistema de diálogos explicativos del videojuego en GameMaker.

En esta sección se presenta la programación del sistema de diálogos utilizado dentro del videojuego. Este componente es fundamental porque permite mostrar al jugador mensajes explicativos, instrucciones y contexto antes de realizar las actividades o minijuegos.

En la imagen se observa un objeto de diálogo en GameMaker, el cual utiliza eventos como Create, Step y Draw GUI. Estos eventos permiten inicializar los textos, controlar su avance y mostrarlos en pantalla de forma clara para el usuario.

La programación evidencia que los diálogos se agregan de manera ordenada mediante instrucciones que van almacenando cada mensaje. Esto permite construir una conversación progresiva con el jugador, donde primero se saluda, luego se introduce el concepto educativo y finalmente se dan las indicaciones necesarias para avanzar.

Durante las pruebas se verificó que:

- Los diálogos se cargan correctamente al iniciar la interacción.
- Los mensajes aparecen en el orden definido.
- El texto es claro y guía al jugador antes de iniciar la actividad.
- El sistema permite explicar conceptos como arrays, posiciones y selección de elementos.
- No se presentan errores críticos al mostrar los mensajes en pantalla.

Este sistema es importante porque conecta la parte educativa con la jugabilidad. Gracias a los diálogos, el jugador no entra al minijuego sin contexto, sino que primero recibe una explicación sencilla que le ayuda a entender qué debe hacer y qué concepto está practicando.

Validación de la interfaz y ejecución general del videojuego

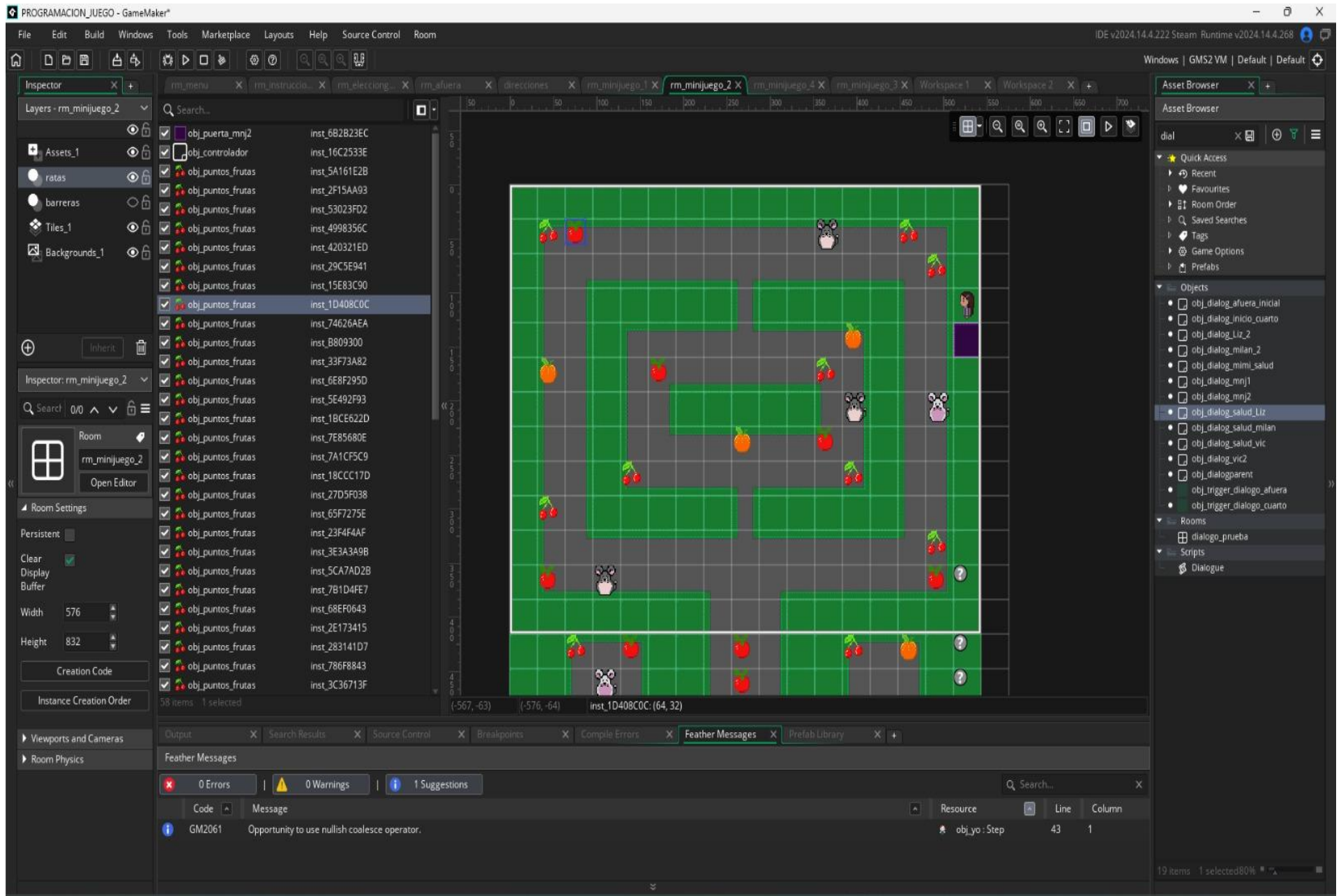


Figura 13. Visualización del entorno de ejecución del videojuego en GameMaker.

En esta sección se presenta la visualización del videojuego en su entorno de ejecución dentro de GameMaker, donde se integran todos los elementos desarrollados durante las etapas de diseño y programación. Esta vista permite observar el funcionamiento general del sistema en un escenario real.

En la imagen se evidencia el mapa del juego con una estructura tipo laberinto, donde el jugador puede desplazarse a través de diferentes caminos. Dentro del entorno se identifican múltiples elementos interactivos, como objetos recolectables (frutas), enemigos (ratones) y zonas específicas del escenario, lo que demuestra la integración de los componentes previamente desarrollados.

Además, se observa la correcta disposición de los recursos dentro del entorno de desarrollo, incluyendo capas, objetos y elementos visuales organizados en el editor. Esto confirma que el videojuego no solo está programado, sino también estructurado de forma adecuada para su ejecución.

Durante las pruebas realizadas, se verificó que:

- El escenario se carga correctamente dentro del motor de desarrollo
- Los elementos visuales (objetos, personajes y recursos) se muestran en pantalla
- El mapa permite el desplazamiento del jugador dentro de las rutas definidas
- Los objetos interactivos se encuentran correctamente ubicados
- El sistema integra de forma adecuada la lógica del juego con la representación visual
- No se presentan errores críticos en la ejecución general del entorno

Esta visualización es clave porque permite comprobar que todos los componentes del videojuego funcionan de manera conjunta, desde la interfaz hasta la lógica interna del sistema. No se trata solo de elementos individuales, sino de un entorno completo en funcionamiento.